

DocuMind AI

Adaptive Multimodal RAG for Source-Grounded PDF Intelligence

Supervisor	Eng / Jana Hatem
Assistant Engineer	Gad Amr
Team	Kemet AI
Deliverables	Documentation, Presentation, Web Application

Built around trust, evidence, hallucination control, and education.

DocuMind AI: Academic Project Documentation

Cover Information

- Project Title: DocuMind AI
- Project Subtitle: Adaptive Multimodal Retrieval-Augmented Generation for Source-Grounded PDF Intelligence
- Supervisor: Eng / Jana Hatem
- Assistant Engineer: Gad Amr
- Team: Kemet AI
- Application Domain: Educational technology, academic document intelligence, PDF question answering, and automated exam generation
- Deployment Target: Railway cloud platform
- Primary Technologies: FastAPI, React, TypeScript, PostgreSQL, FAISS, BM25, Groq LLM, Docker

Abstract

DocuMind AI is a production-oriented adaptive multimodal Retrieval-Augmented Generation system designed to support trustworthy interaction with uploaded PDF documents. The platform enables users to upload academic or technical PDF sources, ask natural-language questions, inspect supporting evidence, and generate structured exams from the same source material. Unlike generic chatbot systems, DocuMind AI is intentionally constrained to answer only from the uploaded PDFs. When the source does not contain enough evidence, the system refuses to fabricate an answer and recommends consulting a doctor or a qualified specialist depending on the domain.

The system combines a React-based frontend, a FastAPI backend, authentication, PostgreSQL persistence, runtime FAISS and BM25 indexes, query rewriting, evidence retrieval, answer grounding, hallucination risk estimation, and structured exam generation. It is deployed as a Railway-ready full-stack application where the frontend and backend can be served from one production container. The project is especially suitable for university doctors, teaching assistants, students, and researchers who need accurate PDF-based answers and source-grounded assessment material.

1. Introduction

Academic knowledge is often stored in PDF documents such as lecture notes, medical guidelines, research papers, technical manuals, and textbooks. These PDFs usually contain more than plain text. They may include tables, figures, scanned pages, charts, diagrams, references, and multi-page explanations. Traditional text-only document question-answering systems are limited

because they often flatten the document into text chunks and ignore layout, page structure, visual context, and tabular relationships.

Large Language Models can produce fluent answers, but fluency alone is not enough in educational or medical contexts. A system may sound confident while relying on information that is not present in the uploaded source. This creates a hallucination risk. In academic workflows, the user needs to know not only the final answer but also where the answer came from and whether the document actually supports it.

DocuMind AI addresses this problem by combining retrieval, source-grounded generation, evidence citation, refusal behavior, and exam generation. The system treats the uploaded PDF as the only factual source. It retrieves relevant evidence, sends only that evidence to the LLM, checks answer support, and exposes reliability signals to the frontend.

2. Problem Statement

The project addresses the following problems:

- 1 Many PDF question-answering tools lose information when documents include tables, figures, scanned pages, or diagrams.
- 2 Generic LLM chat systems may answer from general knowledge instead of the uploaded PDF.
- 3 Users cannot easily verify whether an answer is grounded in the source.
- 4 Medical and academic questions require a safer fallback when evidence is insufficient.
- 5 Educators need a fast way to generate exams from source material while preserving answer keys and references.
- 6 Cloud deployment can lose in-memory indexes after restart if uploaded sources are not persisted.

The main research and engineering challenge is to design a system that remains practical for deployment while improving trust, traceability, and educational usefulness.

3. Project Objectives

The main objectives of DocuMind AI are:

- Build a web platform that allows authenticated users to upload PDF files.
- Generate runtime indexes that support semantic and keyword retrieval.
- Rewrite and classify user queries before retrieval.
- Answer questions using only uploaded PDF content.
- Provide source pages and evidence blocks for every supported answer.
- Estimate hallucination risk and groundedness score.
- Refuse unsupported questions instead of fabricating answers.

- Generate structured exams from uploaded sources.
- Persist uploaded PDFs in PostgreSQL so indexes can be rebuilt after server restart.
- Provide a responsive frontend suitable for desktop and mobile users.
- Deploy the full-stack system on Railway using Docker.

4. Target Users and Use Cases

User Type	Main Use Case	Expected Benefit
Student	Upload lecture notes and ask questions	Faster studying with source-backed answers
University doctor	Generate exams from lecture PDFs	Saves time and preserves academic alignment
Teaching assistant	Prepare quizzes and answer keys	Structured assessment generation
Researcher	Inspect papers and technical documents	Traceable summaries and evidence
Admin / developer	Deploy and maintain the system	Production-ready architecture

5. Functional Requirements

Requirement	Description
User authentication	Users can sign up, log in, and receive JWT tokens.
PDF upload	Users can upload one or more PDF files per chat/source.
Index building	The backend builds runtime retrieval indexes from uploaded PDFs.
Question answering	Users can ask questions against the uploaded source.
Evidence display	The system returns page-level evidence and citations.

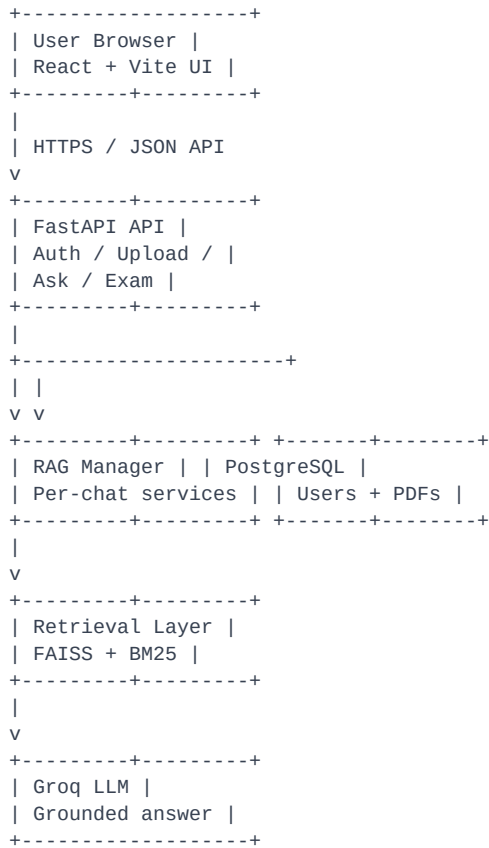
Source-only policy	The LLM must not answer from outside the uploaded source.
Refusal behavior	Unsupported questions receive a safe refusal response.
Exam generation	Users can generate structured exams from the uploaded PDF.
Persistence	Uploaded PDFs are stored in PostgreSQL for rebuild after restart.
Voice input	The frontend supports browser-based speech recognition when available.

6. Non-Functional Requirements

Requirement	Description
Reliability	The system should recover from server restarts by rebuilding indexes.
Security	User access must be protected using JWT authentication.
Traceability	Answers must include evidence and source page references.
Maintainability	Backend and frontend responsibilities must be separated clearly.
Scalability	The architecture should allow future migration to pgvector or Qdrant.
Usability	The frontend should be responsive and mobile friendly.
Deployability	The system should run in a Docker container on Railway.

7. Overall System Architecture

The platform is structured as a full-stack web application. The frontend handles user interaction, while the backend handles authentication, document ingestion, retrieval, generation, persistence, and API responses.



8. Backend Architecture

The backend follows a modular FastAPI structure. Each module has a clear responsibility.

```

backend/documind_rag/
app/
main.py Application factory and router registration
schemas.py Pydantic request/response contracts
controllers/ HTTP endpoints
core/ Config, database, security, dependencies
models/ SQLAlchemy database models
repositories/ Database access layer
services/ Auth and business services
rag/
service.py Production facade around RAG pipeline
pipeline.py Pipeline exports
notebook_core.py Notebook-derived RAG logic

```

8.1 Backend Component Responsibilities

Component	Responsibility
main.py	Creates the FastAPI app, registers routers, serves frontend files.
schemas.py	Defines stable API contracts using Pydantic.

auth_controller.py	Handles signup, login, and current-user dependency.
document_controller.py	Handles PDF upload, storage, and index build.
assistant_controller.py	Handles source-grounded question answering.
exam_controller.py	Handles source-grounded exam generation.
index_rebuild.py	Rebuilds runtime indexes from persisted PDFs after restart.
database.py	Configures SQLAlchemy engine and table creation.
security.py	Handles password hashing and JWT utilities.
rag/service.py	Wraps the RAG pipeline behind a production API.

9. Frontend Architecture

The frontend is implemented with React, Vite, TypeScript, and Tailwind CSS. It communicates with the backend through typed API service functions.

```

frontend/src/
pages/
  LandingPage.tsx
  AuthPage.tsx
  DocumentsPage.tsx
  AssistantPage.tsx
  ExamStudioPage.tsx
  AnalyticsPage.tsx
services/
  api.ts
  auth.ts
components/
  layout/
  assistant/
  documents/
  ui/
types/
index.ts

```

9.1 Frontend Features

- Landing page with mobile horizontal card rails.
- Dark/light theme toggle.
- Language preference toggle.
- Authentication screens.

- Document upload workspace.
- Chat assistant with evidence display.
- Mobile evidence drawer.
- Browser voice input through Web Speech API.
- Exam Studio configuration and result view.
- Team section with member photos.

10. Database and Persistence Architecture

The production database stores user accounts and uploaded source PDFs. Runtime vector indexes are not stored permanently; instead, source PDFs are persisted and used to rebuild indexes when needed.

```
+-----+ +-----+
| users | | document_sources |
+-----+ +-----+
| id |<-----| user_id |
| name | | chat_id |
| email | | filename |
| password_hash | | content (PDF bytes) |
| created_at | | created_at |
+-----+ +-----+
```

10.1 Persistence Workflow

- 1 User uploads a PDF.
- 2 Backend writes the PDF to a local runtime upload directory.
- 3 Backend builds FAISS/BM25 indexes.
- 4 Backend stores the PDF bytes in PostgreSQL.
- 5 If the server restarts, runtime indexes disappear.
- 6 On the next /ask or /exam request, the backend detects the missing index.
- 7 The backend reloads the PDF bytes from PostgreSQL.
- 8 The backend rebuilds indexes and retries the original request.

This strategy keeps the hackathon deployment lightweight while solving the restart problem.

11. RAG Pipeline Architecture

The RAG pipeline is designed around evidence-first generation. The system retrieves and verifies before presenting an answer.

```
User Question
|
v
```



```
Query Rewrite
|
v
Query Classification
|
v
Hybrid Retrieval
FAISS + BM25
|
v
Reranking
|
v
Context Builder
|
v
Groq LLM Generation
|
v
Citation + Claim Verification
|
v
Structured Answer Response
```

11.1 Query Rewriting

Query rewriting improves retrieval by normalizing the user question and generating search-oriented query variants. This is useful when the user asks vague or conversational questions. The rewritten query is returned in the API response for transparency.

11.2 Query Classification

The system classifies questions into query types such as factual text, table question, image/chart question, summarization, or cross-page question. This classification helps choose the most appropriate retrieval path.

11.3 Hybrid Retrieval

The system combines:

- FAISS: Fast semantic vector search.
- BM25: Keyword-based sparse retrieval.

This hybrid strategy improves recall because semantic search can capture meaning, while BM25 preserves exact terms and keywords.

11.4 Reranking and Context Building

Retrieved candidates are reranked before generation. The context builder then assembles the most relevant chunks into a prompt-ready evidence package. This reduces noise and helps the LLM focus on supported content.

11.5 Grounded Generation

The LLM receives only the retrieved context and must answer from that context. The system prompt explicitly prevents using outside information.

12. Source-Only Safety Design

DocuMind AI is intentionally designed to avoid unsupported answers. If the uploaded PDF does not contain enough evidence, the backend returns a refusal message instead of allowing the LLM to guess.

The refusal behavior is domain-aware:

- If the question is medical, the user is advised to consult a doctor.
- If the question belongs to another domain, the user is advised to consult a qualified specialist.
- The user may also upload a clearer or more relevant source.

This design is important for educational and medical use cases where incorrect information can cause harm.

13. Hallucination Risk and Groundedness

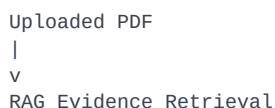
The system exposes answer quality through structured metrics:

Metric	Meaning
Confidence	Estimated reliability of the answer.
Groundedness score	How strongly the answer is supported by source evidence.
Hallucination risk	Estimated fraction of unsupported claims.
Unsupported claims	Number of claims that could not be verified.
Evidence list	Retrieved chunks supporting the answer.
Sources	Page numbers used by the response.

These metrics help the user evaluate whether the answer should be trusted.

14. Exam Studio Architecture

Exam Studio extends the RAG system from question answering into assessment generation. It uses the uploaded source as the only factual basis for exam questions.



```
|  
v  
Exam Prompt  
|  
v  
Groq LLM  
|  
v  
Strict JSON Exam Output  
|  
v  
Pydantic Validation  
|  
v  
Frontend Exam Cards
```

14.1 Exam Configuration

Users can configure:

- Source document.
- Question count.
- Difficulty.
- Question formats.
- Optional topic focus.

14.2 Exam Output Schema

Each generated question includes:

- Question type.
- Question text.
- Options when applicable.
- Correct answer.
- Explanation.
- Source page.
- Source section.

The response also includes groundedness score, hallucination risk, evidence, and source pages.

15. API Design

The API is designed around stable Pydantic schemas to support frontend integration and future backend/frontend separation.

Endpoint	Method	Description
/auth/signup	POST	Creates a new user account.
/auth/login	POST	Authenticates user and returns JWT.
/health	GET	Returns current service readiness.
/chats/{chat_id}/sources	POST	Uploads PDFs and builds indexes.
/ask	POST	Answers a question from uploaded source.
/exam	POST	Generates a structured exam from uploaded source.
/chats/{chat_id}/memory	GET	Returns bounded conversation memory.

16. Deployment Architecture

The project is deployed on Railway using Docker. The root Dockerfile builds the frontend and backend into one production container.

```

Docker build
|
+--> Node stage builds Vite frontend
|
+--> Python stage installs FastAPI backend
|
+--> Frontend dist copied into backend image
|
v
Railway service
|
+--> Serves FastAPI endpoints
+--> Serves React SPA
+--> Connects to PostgreSQL

```

16.1 Required Environment Variables

Variable	Purpose
GROQ_API_KEY	LLM provider API key.
DATABASE_URL	PostgreSQL connection string.
JWT_SECRET_KEY	Secret for JWT signing.

ACCESS_TOKEN_EXPIRE_MINUTES	Token lifetime.
DOCUMIND_CORS_ORIGINS	Allowed frontend origins.
DOCUMIND_LOW_MEMORY	Enables Railway-friendly low-memory mode.
DOCUMIND_MAX_ANSWER_TOKENS	Controls maximum generated answer length.

17. Security Considerations

The platform includes the following security-related design decisions:

- Passwords are stored as hashes, not plain text.
- JWT tokens protect authenticated routes.
- Uploaded sources are associated with the authenticated user.
- Backend routes validate request bodies through Pydantic.
- Secrets are configured through environment variables.
- The frontend does not contain the Groq API key.

Future security improvements should include upload size limits, malware scanning for files, rate limiting, role-based permissions, and audit logs.

18. Evaluation and Testing

Current validation includes:

- Python syntax compilation.
- TypeScript compilation.
- Vite production build.
- Railway health checks.
- Manual production smoke tests.
- PDF question-answering checks using known source documents.

Recommended evaluation improvements:

- Automated API tests for authentication, upload, ask, and exam endpoints.
- Playwright end-to-end tests for the frontend.
- A benchmark set of PDFs with expected answers.
- Groundedness evaluation across multiple domains.
- Regression tests for unsupported-question refusal.

19. Current Strengths

DocuMind AI has several strengths:

- Strong separation between frontend, backend, database, and RAG logic.
- Production deployment through Docker and Railway.
- Source-only answer policy.
- Evidence-based answer presentation.
- Persistent uploaded source storage.
- Automatic index rebuild after restart.
- Structured Pydantic response schemas.
- Exam generation for academic use.
- Responsive mobile user interface.
- Voice input support.

20. Limitations

The current implementation also has limitations:

- FAISS indexes are rebuilt in memory rather than persisted as vector indexes.
- Rebuild time increases with larger PDFs.
- Browser voice input depends on Web Speech API support.
- Full Arabic/English translation is not yet complete.
- OCR quality depends on the quality of scanned PDFs.
- More automated tests are needed before large-scale production usage.

21. Future Work

Future work should focus on moving the system from hackathon-ready to scalable academic product.

21.1 Technical Future Work

- Add a persistent vector database option such as pgvector or Qdrant.
- Add background indexing jobs for large PDFs.
- Add live progress streaming during PDF processing.

- Add scalable object storage for uploaded PDFs.
- Add role-based access control for teacher, student, and admin accounts.
- Add full Arabic and English localization.
- Add better OCR preprocessing for scanned PDFs.
- Add server-side speech-to-text for more reliable voice mode.

21.2 Academic and Product Future Work

- Add exam export to PDF, DOCX, and printable formats.
- Add saved exam banks per course.
- Add classroom/team workspaces.
- Add multi-document comparison across several PDFs.
- Add learning analytics for student usage.
- Add instructor review mode for generated questions.
- Add rubric generation and marking support.

21.3 Evaluation Future Work

- Build a dataset of source PDFs with verified question-answer pairs.
- Evaluate retrieval recall, answer groundedness, citation accuracy, and refusal accuracy.
- Add hallucination regression tests.
- Measure indexing time and answer latency across different PDF sizes.
- Compare FAISS-only, BM25-only, and hybrid retrieval performance.

22. Conclusion

DocuMind AI demonstrates a practical and academically relevant approach to trustworthy PDF intelligence. The system does not simply chat with documents; it retrieves evidence, verifies support, refuses unsupported questions, and generates structured educational content. By combining source-only RAG, hallucination control, persistent uploaded sources, and Exam Studio, the project provides a strong foundation for educational AI workflows.

The architecture is suitable for a hackathon demonstration and has a clear path toward larger-scale production through persistent vector storage, stronger evaluation, background jobs, and full localization.

References

This documentation is based on the implemented project source code and internal project files:

- README.md
- Dockerfile
- railway.json
- backend/documind_rag/app/main.py
- backend/documind_rag/app/schemas.py
- backend/documind_rag/app/controllers/
- backend/documind_rag/app/models/
- backend/documind_rag/app/repositories/
- backend/documind_rag/rag/service.py
- backend/documind_rag/rag/notebook_core.py
- frontend/src/pages/
- frontend/src/services/api.ts
- docs/PROJECT_DOCUMENTATION.md